Hamster's Stash 🐹

Hamster's Stash — 開發規格書

Flutter · Riverpod · Isar · TDD · CI/CD

1. 核心願景 (Core Vision)

- 極致隱私：100% 本地儲存 (On-device)，無雲端資料同步，消除資料外洩疑慮。
- 投資友善：專為玩美股與台股的使用者設計，即時追蹤市值。
- 聰明匯率：提供建議匯率，但保留手動修正權限，精準記錄換匯成本。
- 可愛安全感：以「保險箱」為主題的可愛介面，結合生物辨識鎖。

2. 技術棧 (Tech Stack)

類別	技術	說明
Framework	Flutter 3.2x+	跨平台 UI
State Management	Riverpod	可測試性高、支援 code generation
Database	Isar Database	本地加密資料庫，效能優於 SQLite
ML Engine	google_ml_kit	用於掃描台灣發票 QR Code
Stock API	Yahoo Finance API	透過 yahoofin 或 HTTP 請求
Auth	local_auth	支援 FaceID / 指紋

3. 資料模型 (Data Schema)

Account (帳戶)

欄位	型別	說明
id	Id	主鍵
name	String	如: Firstrade, 國泰世華
type	Enum	Cash, Bank, Investment, CreditCard, RealEstate, Vehicle, Insurance, Other
currency	String	TWD, USD
assetTerm	Enum	current (短期) / nonCurrent (長期)
stockSymbols	List<String>	該帳戶持有的股票代碼

Transaction (交易紀錄)

欄位	型別	說明
id	Id	主鍵
type	Enum	expense / income / transfer
amount	Double	原始交易金額
currency	String	交易幣別
exchangeRate	Double	最終使用的匯率
isManualRate	Boolean	是否由使用者手動輸入
categoryId	Int	大分類 ID
subcategoryId	Int?	小分類 ID (可選)
accountId	Id	所屬帳戶
toAccountId	Id?	轉帳目標帳戶 (僅 transfer)
dateTime	DateTime	交易時間
note	String?	備註
tags	List<String>	自定義標籤
photoPath	String?	收據照片本地路徑
stockSymbol	String?	股票代碼 (AAPL, 2330.TW)
invoiceNumber	String?	發票號碼
recurringId	Id?	關聯週期性交易 ID

Category (分類)

欄位	型別	說明
id	Id	主鍵
name	String	分類名稱 (如: 飲食、交通)
icon	String	emoji 或 icon name
color	String	Hex 顏色碼
type	Enum	expense / income
parentId	Id?	父分類 ID (null = 大分類)
sortOrder	Int	排序順序
isDefault	Boolean	是否為系統預設

Budget (預算)

欄位	型別	說明
id	Id	主鍵
categoryId	Id?	綁定分類 (null = 總預算)
amount	Double	預算金額
period	Enum	weekly / monthly / yearly
carryOver	Boolean	是否結轉餘額
alertThreshold	Double	警告閾值 (0.0–1.0，預設 0.8)

RecurringRule (週期性規則)

欄位	型別	說明
id	Id	主鍵
templateTransaction	Transaction	交易模板 (含金額、分類、帳戶)
frequency	Enum	daily / weekly / monthly / yearly / custom
customDays	Int?	自定天數 (僅 custom)
startDate	DateTime	開始日期
endDate	DateTime?	結束日期 (可選)
maxOccurrences	Int?	最大執行次數 (可選)
isActive	Boolean	是否啟用

4. 基礎記帳功能 (Core Bookkeeping Features)

A. 大小分類系統 (Hierarchical Categories)

支援兩層式分類架構，使用者可自由新增、編輯、刪除與排序大分類與小分類。

- 大分類 **(Parent)**：如飲食、交通、娛樂、購物、住居、醫療、教育、投資、薪資、其他。
- 小分類 **(Child)**：如飲食 → 早餐/午餐/晚餐/飲料/孝食；娛樂 → 電影院/KTV/遊戲/訂閱制。
- 自定義：使用者可新增自己的大小分類，每個分類含自定 icon (支援 emoji) 與顏色。
- 預設分類：首次安裝時內建一組預設分類，使用者可隨時修改或刪除。
- 收入/支出分開：收入與支出各有獨立的分類樹，避免混淆。

B. 記帳流程 (Transaction Entry)

- 快速記帳：點擊金額 → 選大分類 → 選小分類 → 完成，最少 3 步即可記一筆帳。
- 內建計算機：金額欄位支援基本四則運算 (+−×÷)，如輸入 150+80 自動算出 230。
- 收入/支出/轉帳：三種交易類型一鍵切換，轉帳需選擇來源與目標帳戶。
- 附加資訊：每筆交易可附加備註、照片 (收據拍照)、標籤 (Tag)。
- 日期編輯：預設為今天，可回溯修改日期以補記過去的消費。

C. 預算管理 (Budget Management)

- 分類預算：可針對每個大分類或小分類設定每月/每週預算上限。
- 總預算：支援設定整體每月總支出上限。
- 進度條：預算使用比例以進度條視覺化呈現，超過 80% 變色警告，100% 爭紅。
- 超支提醒：透過本地通知在接近或超過預算時提醒使用者。
- 餘額結轉：可選擇是否將未用完的預算餘額結轉至下一期。

D. 週期性交易 (Recurring Transactions)

- 自動記帳：支援設定重複週期：每日、每週、每月、每年、自定天數。
- 適用情境：房租、訂閱制 (Netflix, Spotify)、保險費、薪資入帳、固定存款等。
- 結束條件：可設定結束日期或重複次數上限，到期自動停止。
- 提前通知：在週期性交易執行前一天發送本地通知提醒。

E. 帳戶間轉帳 (Account Transfers)

- 內部轉帳：支援帳戶間資金移動，如從「國泰世華」轉到「Firstrade」。
- 跨幣轉帳：當來源與目標帳戶幣別不同時，自動帶入匯率換算。
- 不計入收支：轉帳交易不納入收入/支出統計，僅影響帳戶餘額。

F. 報表與圖表 (Reports & Charts)

- 圓餅圖：按分類顯示支出/收入佔比，可點擊展開小分類明細。
- 長條圖：每月/每週收支對比，直觀看出花費趨勢。
- 折線圖：結餘/淨資產趨勢，支援自定時間區間。
- 分類排名：顯示花費最高的 Top N 分類，協助找出消費習慣。
- 月/週/年報表：支援以月、週、年為單位的報表切換。

G. 搜尋與篩選 (Search & Filter)

- 關鍵字搜尋：支援依備註、分類名稱、帳戶名稱搜尋交易紀錄。
- 篩選條件：依日期範圍、金額範圍、分類、帳戶、 標籤多條件組合篩選。
- 排序：支援依日期、金額進行升/降序排列。

H. 日曆檢視 (Calendar View)

- 月曆模式：在月曆上顯示每日總支出/總收入，讓使用者一眉掌握花費分佈。
- 點擊展開：點擊某日可展開該日所有交易明細。
- 色彩標記：依支出高低顯示不同深淺的顏色標記 (heatmap 風格)。

I. 資料匯出 (Data Export)

- **CSV** 匯出：將交易紀錄匯出為 CSV 檔案，可導入 Excel 或 Google Sheets 進行進階分析。
- 篩選匯出：支援依時間區間、帳戶、 分類篩選後再匯出。
- 分享方式：透過系統 Share Sheet 分享至 Files、AirDrop、Email 等。

J. 桌面小工具 (Home Screen Widget)

- 快速記帳 **Widget**：在手機桌面直接點擊即可開啟記帳頁面，免開 App 主畫面。
- 餘額總覽 **Widget**：顯示當月支出/餘額/預算概況，一眉掌握財務狀況。
- 平台支援：iOS WidgetKit + Android App Widgets，分別實作。

---

5. 進階功能需求 (Advanced Features)

A. 跨國資金與匯率 (Multi-Currency & FX)

- 自動建議：當交易幣別與帳戶幣別不符時，自動串接 frankfurter.app 或 Yahoo Finance 抓取即時匯率。
- 手動覆蓋：UI 必須顯示推薦匯率，但允許使用者直接修改。若使用者修改過，則停止自動更新。

B. 股市行情追蹤 (Market Data)

- 格式自動識別：輸入 TSLA → 識別為美股；輸入 2330 → 自動轉換為 2330.TW (台股)。
- 現值計算：資產總覽頁面顯示 (持股數 × 即時股價) × 匯率。

C. 發票掃描 (Invoice Scanner)

- 本地解析：解析台灣電子發票 QR Code，索引發票號碼與十六進位金額 (Hex to Decimal)。
- 隱私：掃描過程不將影像上傳至伺服器。

D. 安全鎖 (Privacy Lock)

- 冷啟動保護：每次 App 由背景轉回前景或冷啟動時，強制觸發生物辨識 (FaceID/Fingerprint)。

E. 應收/應付帳款 (Receivables & Payables)

支援使用者記錄「別人欠我的」與「我欠別人的」債務關係，適用於朋友分帳、借貸、預付款等情境。

- 應收帳款 **(Receivable)**：記錄他人尚未歸還的款項，含對象姓名、金額、幣別、到期日 (可選)。
- 應付帳款 **(Payable)**：記錄自己尚未支付的款項，同樣含對象、金額、幣別、到期日。
- 狀態追蹤：每筆應收/應付帳款有 pending / partial / settled 三種狀態，部分收款時可記錄多次收付。
- 自動沖銷：當狀態變更為 settled 時，可選擇自動產生對應的 Transaction 記錄。
- 到期提醒：支援本地通知 (flutter_local_notifications)，在到期日前提醒使用者。
- 總覽整合：資產總覽頁顯示淨應收/應付餘額，納入淨資產計算。

F. 資產分類：短期與長期 (Asset Classification: Current & Non-Current)

將使用者的資產依流動性分為「短期資產 (Current Assets)」與「長期資產 (Non-Current Assets)」，提供更完整的財務健康全貌。

- 短期資產 **(Current)**：可在一年內變現的資產，包含：
  - 現金與活存：現金帳戶、活期儲蓄帳戶

  - 流動投資：股票帳戶 (Firstrade 等)、短期債券、貨幣市場基金
  - 應收款項：一年內到期的應收帳款
  - 預付費用：已預付但尚未使用的費用 (如年繳保費)
- 長期資產 **(Non-Current)**：持有超過一年的資產，包含：
  - 定存/儲蓄險：定期存款、儲蓄型保單
  - 不動產：房屋、土地 (手動輸入現值或定期更新)
  - 長期投資：ETF 長期持倉、退休金帳戶、勞保自提
  - 其他：車輛、珍貴物品等有價資產
- 帳戶分類標記：Account 新增 assetTerm 欄位 (current / nonCurrent)，建立帳戶時指定，也可事後變更。
- 總覽頁呈現：資產總覽頁分兩區塊顯示：短期資產小計、長期資產小計，加總為總資產。
- 淨資產計算：總資產 − 總負債 (信用卡帳單 + 應付帳款) = 淨資產，並以折線圖追蹤淨資產歷史變化。
- 手動估值：不動產、車輛等無即時報價的資產，提供「更新現值」功能，記錄估值日期與金額。
- 資產配置圓餅圖：在報表頁顯示短期 vs 長期資產佔比，以及各類別明細 (現金/股票/不動產...)。

---

6. 平台特定設定 (Platform Configuration)

iOS (Info.plist)

```
NSCameraUsageDescription: 我們需要相機來掃描發票並自動為您填寫金額。
NSFaceIDUsageDescription: 我們使用 FaceID 保護您的保險箱隱私。
UIFileSharingEnabled: true
LSSupportsOpeningDocumentsInPlace: true
```

Android (AndroidManifest.xml)

```
android.permission.CAMERA
android.permission.USE_BIOMETRIC
android.permission.POST_NOTIFICATIONS  // 預算超支、週期交易、應收應付到期提醒
```

以下為新增開發最佳實踐章節

7. 專案架構與目錄結構 (Project Architecture)

採用 Feature-First 架構，每個功能模組內含自己的 data / domain / presentation 三層，確保單一職責原則 (SRP) 與可測試性。

```
lib/
  core/                # 共用工具、常數、擴展方法
    constants/         # API URLs, 遊定參數
    errors/            # 自定義 Exception 類別
    extensions/        # Dart extension methods
    utils/             # 格式化、轉換工具
  features/
    accounts/
      data/            # Repository impl, Data sources, DTOs
      domain/          # Entities, Repository interface, Use cases
      presentation/    # Screens, Widgets, Riverpod providers
    transactions/
    categories/          # 大小分類 CRUD 與選擇器
    budget/              # 預算設定、進度追蹤、超支提醒
    recurring/           # 週期性交易規則與排程
    reports/             # 圓餅圖、長條圖、折線圖、分類排名
    calendar/            # 月曆檢視與每日摘要
    search/              # 搜尋、篩選、排序
    data_export/         # CSV 匯出與分享
    asset_overview/      # 資產總覽、短期/長期分類、淨資產計算
    market_data/
    invoice_scanner/
    exchange_rate/
    receivables_payables/
    auth_lock/
  app.dart             # MaterialApp 設定
  main.dart            # Entry point
test/
  unit/                # 純邏輯測試
  widget/              # UI 元件測試
  integration/         # 整合測試
  e2e/                 # 端對端測試
  mocks/               # 共用 mock 與 fake
  fixtures/            # 測試用 JSON / 圖片等固定資料
```

8. 測試策略與 TDD (Testing Strategy & TDD)

8.1 TDD 流程 (Red-Green-Refactor)

本專案強制採用 Test-Driven Development，所有功能皆需先寫測試再實作：

- **Red:** 先寫一個失敗的測試，明確定義預期行為。
- **Green:** 用最小的程式碼讓測試通過。
- **Refactor:** 重構程式碼，確保測試仍然通過。

8.2 測試金字塔 (Testing Pyramid)

目標總體測試覆蓋率：≥80%，分層配比如下：

層級	比例	工具	測試內容
Unit Test	~60%	flutter_test, mockito, mocktail	純函式、Use Cases、Repository、Providers、資料轉換
Widget Test	~20%	flutter_test (WidgetTester)	UI 元件渲染、互動、狀態變化
Integration Test	~15%	integration_test	跨層流程：輸入 → 儲存 → 顯示
E2E Test	~5%	patrol / integration_test (device)	完整用戶旅程，含生物辨識 mock

8.3 各功能測試重點

A. 分類系統 (Categories)

- **Unit:** Category CRUD、父子關係正確性、刪除大分類時小分類聯動處理
- **Widget:** 分類選擇器 UI：先顯示大分類→點擊展開小分類、自定 icon 顯示
- **Integration:** 新增自定分類 → 記帳時可選擇 → 報表正確歸類

B. 預算管理 (Budget)

- **Unit:** 預算計算邏輯、超支判斷、餘額結轉邏輯、alertThreshold 觸發條件
- **Widget:** 進度條顏色變化 (>80% 警告、>100% 爭紅)、預算設定表單
- **Integration:** 設定預算 → 記帳超過閾值 → 通知觸發 → UI 更新

C. 週期性交易 (Recurring)

- **Unit:** 下次執行日期計算、結束條件判斷、自定天數週期
- **Widget:** 週期性交易列表顯示、編輯/暫停/刪除操作
- **Integration:** 建立每月規則 → 模擬時間前進 → 驗證自動產生 Transaction

D. 帳戶轉帳與報表 (Transfers & Reports)

- **Unit:** 轉帳後雙方帳戶餘額變化、跨幣轉帳匯率計算、轉帳不計入收支
- **Widget:** 圓餅圖/長條圖/折線圖渲染正確性、時間區間切換
- **Integration:** 記帳多筆 → 切換報表檢視 → 驗證圓餅圖比例正確

E. 匯率模組 (Exchange Rate)

- **Unit:** ExchangeRateRepository 的 API 回應解析、快取逾期、手動匯率覆蓋邏輯
- **Widget:** 匯率輸入框顯示推薦值、編輯後狀態變更、isManualRate 正確設定
- **Integration:** 新增交易 → 匯率自動帶入 → 修改 → 儲存 → 重新打開驗證

F. 股市追蹤 (Market Data)

- **Unit:** 股票代碼格式識別 (TSLA → US, 2330 → 2330.TW)、現值計算公式
- **Widget:** 資產總覽卡片顯示正確數值、loading / error 狀態
- **Integration:** 新增股票帳戶 → API 回傳 mock 股價 → 總覽頁正確顯示

G. 發票掃描 (Invoice Scanner)

- **Unit:** QR Code 解析器：各種發票格式、Hex 轉 Decimal、異常格式處理
- **Widget:** 相機預覽當權限被拒絕時的 UI fallback
- **E2E:** 模擬掃描圖片 → 自動填入金額 → 儲存交易

H. 生物辨識鎖 (Auth Lock)

- **Unit:** AuthService 的狀態機：locked / unlocked / unavailable
- **Widget:** 鎖定畫面在認證失敗時不應顯示資料
- **Integration:** App lifecycle (resume) 觸發重新驗證流程

I. 應收/應付帳款 (Receivables & Payables)

- **Unit:** 狀態機轉換 (pending → partial → settled)、settledAmount 計算、淨應收/應付彙總
- **Unit:** 自動沖銷產生 Transaction 的金額、幣別、關聯 ID 正確性
- **Widget:** 應收/應付列表的篩選 (pending/settled)、到期日標記顯示
- **Integration:** 新增應收 → 部分收款 → 狀態變更 → 總覽頁數字更新
- **Integration:** 到期提醒通知觸發流程 (mock flutter_local_notifications)

J. 資產分類 (Asset Classification)

- **Unit:** 總資產計算：短期小計 + 長期小計 = 總資產、淨資產 = 總資產 − 總負債
- **Unit:** ManualValuation CRUD、最新估值取得邏輯、估值歷史排序
- **Widget:** 總覽頁短期/長期區塊正確歸類、資產配置圓餅圖比例
- **Integration:** 新增不動產帳戶 → 輸入估值 → 總覽頁長期資產更新 → 淨資產重算

8.4 測試命名慣例

採用 Given-When-Then 命名法確保可讀性：

```
test('given USD account, when adding TWD transaction, then auto-suggest exchange rate')
test('given manual rate set, when reopening transaction, then rate is preserved')
test('given input 2330, when resolving symbol, then returns 2330.TW')
test('given food category with 3000 budget, when spent 2500, then progress bar shows 83%')
test('given monthly recurring rule, when date advances 1 month, then new transaction created')
test('given transfer between TWD and USD accounts, then neither counts as income or expense')
test('given receivable with partial payment, when settling remainder, then status becomes settled')
test('given payable settled, when auto-reconcile enabled, then transaction is created')
```

```
test('given 2 current and 1 non-current accounts, then overview shows correct subtotals')
test('given real estate account with manual valuation, then net worth includes latest valuation')
```

8.5 Mock 策略

- 網路請求：使用 http_mock_adapter 或 mocktail 封裝 Dio/http client，避免真實 API 呼叫。
- 資料庫：建立 FakeIsarDatabase，在記憶體中運行，無需磁碟 IO。
- 平台服務：local_auth 與 camera 透過 FakeLocalAuth / MockCamera 注入。
- **Riverpod**：使用 ProviderContainer overrides 注入 mock provider。

---

9. CI/CD 流程 (Continuous Integration / Delivery)

9.1 GitHub Actions 建議 Pipeline

Stage	觸發條件	執行內容
Lint & Format	每次 PR / Push	dart analyze + dart format --set-exit-if-changed
Unit & Widget Tests	每次 PR / Push	flutter test --coverage
Coverage Gate	每次 PR	檢查覆蓋率 ≥ 80%，否則失敗
Integration Tests	merge to main	flutter test integration_test/
Build (iOS)	merge to main	flutter build ios --release
Build (Android)	merge to main	flutter build appbundle --release
Deploy to TestFlight / Play Console	Tag 版本號	自動上傳 (fastlane)

9.2 分支策略 (Branching Strategy)

- **main:** 產品版本，只接受通過 CI 的 PR merge。
- **develop:** 開發主線，日常 feature branch 合併目標。
- **feature/*:** 功能分支，命名如 feature/exchange-rate-override。
- **hotfix/*:** 緊急修復，從 main 分出並直接合回。

---

10. 錯誤處理與韌性設計 (Error Handling & Resilience)

10.1 網路層 (Network)

- 重試機制：API 呼叫採用 Exponential Backoff，最多重試 3 次。
- 離線優先：匯率與股價快取至 Isar，網路斷線時顯示上次更新時間與快取資料。
- **Timeout**：所有 HTTP 請求設定 10 秒 timeout，避免 UI 無限等待。

10.2 資料層 (Data)

- 輸入驗證：金額欄位限制數字輸入、小數點最多兩位 (台幣) 或四位 (匯率)。
- 資料序列化：所有 API 回應經過 DTO 轉換，不直接使用 raw JSON。
- 資料庫遷移：Isar schema 變更時提供 migration 路徑，避免使用者資料遺失。

10.3 UI 層

- **AsyncValue**：所有非同步資料統一用 Riverpod 的 AsyncValue<T> 處理 loading / data / error 三種狀態。
- 空狀態：每個列表頁面需設計 Empty State 插畫，符合「可愛保險箱」主題。
- **Snackbar / Toast**：網路錯誤、儲存成功等回饋統一用 Snackbar 呈現。

---

11. 安全性設計 (Security Design)

- **Isar** 加密：啟用 Isar 的內建 AES-256 加密，金鑰存儲於 iOS Keychain / Android Keystore。
- 記憶體清除：App 進入背景時清除敏感 UI 資料 (e.g., 資產總額)，避免 task switcher 截圖洩露。
- **API Key** 保護：使用 --dart-define 注入 API key，不寫死在程式碼中。
- 防範 **Jailbreak/Root**：建議使用 flutter_jailbreak_detection 在啟動時檢測並警告。
- **SSL Pinning**：對 Yahoo Finance 與匯率 API 請求啟用 Certificate Pinning。

---

12. 效能最佳化 (Performance Optimization)

- 懶載入：股價查詢採用 debounce (300ms)，避免頻繁 API 呼叫。
- 分頁載入：交易紀錄列表採用 infinite scroll，每次載入 30 筆。
- **Isar Index**：對 Transaction.dateTime 與 Transaction.categoryId 建立索引。
- **Widget Rebuild**：使用 Riverpod 的 select 與 family 縮小 rebuild 範圍，避免整頁重繪。
- 圖片快取：如有用到股票 Logo，採用 cached_network_image 與本地快取。

---

13. 可觀測性與日誌 (Observability & Logging)

- **Crash Reporting**：整合 Firebase Crashlytics 或 Sentry (選擇 Sentry 則符合隱私優先原則)。
- 本地日誌：使用 logger 套件，Debug mode 輸出詳細日誌，Release mode 只記錄 error 級別。
- **API** 呼叫監控：透過 Dio Interceptor 記錄 request/response 時間與狀態碼。
- 隱私合規：日誌中絕不記錄使用者財務數據、帳戶余額或個人資訊。

---

14. 無障礙設計 (Accessibility)

- **Semantics**：所有資產數字加上 Semantics label，支援 VoiceOver / TalkBack。
- 對比度：所有文字與背景的對比度符合 WCAG 2.1 AA 標準 (≥ 4.5:1)。
- 字型縮放：支援系統字型縮放，不寫死字型大小。
- 觸控區域：所有可點擊區域最小 48x48dp。

15. 資料模型增強建議

15.1 新增欄位

Model	欄位	型別	說明
Account	balance	Double	快取餘額，避免每次重算所有交易
Account	icon	String	自定義帳戶圖示 (emoji 或 icon name)
Account	isArchived	Boolean	軟刪除，保留歷史資料
Transaction	note	String?	使用者備註
Transaction	invoiceNumber	String?	發票號碼 (掃描後自動填入)
Transaction	createdAt	DateTime	建立時間 (audit trail)
Transaction	updatedAt	DateTime	更新時間 (audit trail)
StockHolding	— (new)	Collection	獨立持股記錄: symbol, shares, avgCost, accountId
Receivable/Payable	— (new)	Collection	應收/應付帳款記錄 (見 14.2)

15.2 Receivable / Payable 資料模型

欄位	型別	說明
id	Id	主鍵
type	Enum	receivable (應收) / payable (應付)
counterparty	String	對象姓名 (如: 小明、房東)
totalAmount	Double	總金額
settledAmount	Double	已收/已付金額，預設 0
currency	String	幣別 (TWD, USD)
status	Enum	pending / partial / settled
dueDate	DateTime?	到期日 (可選)
note	String?	備註
linkedTransactionIds	List<Id>	關聯的 Transaction ID (自動沖銷用)
createdAt	DateTime	建立時間
updatedAt	DateTime	更新時間

15.3 ManualValuation (手動估值記錄)

針對無即時報價的長期資產 (不動產、車輛等)，提供定期更新現值的機制。

欄位	型別	說明
id	Id	主鍵
accountId	Id	關聯的帳戶 (RealEstate, Vehicle 等)
valuationAmount	Double	估計現值
currency	String	幣別
valuationDate	DateTime	估值日期
note	String?	估值依據或備註

15.4 Category 預設分類

建議內建預設分類並允許自定義：食物、交通、娛樂、購物、醫療、投資、轉帳、薪資、其他。每個 Category 含 icon 與 color 屬性。

---

16. 版本管理與發佈 (Versioning & Release)

- **Semantic Versioning**：採用 MAJOR.MINOR.PATCH 格式，如 1.2.0+15 (versionName+buildNumber)。
- **Changelog**：維護 CHANGELOG.md，每次版本記錄 Added / Changed / Fixed / Removed。
- **Feature Flags**：新功能透過 local feature flag 控制開關，漸進式上線。
- 定期保養：每季檢查依賴更新 (dependabot 或 renovate)，特別注意 Flutter SDK 與 Isar 版本。

---

17. 開發工作流總覽

以下爲一個 feature 從開始到上線的完整流程：

1. 建立 **Feature Branch** — 從 develop 分出 feature/xxx
2. 寫測試 **(Red)** — 先寫失敗的 unit test 與 widget test
3. 實作功能 **(Green)** — 用最少程式碼讓測試通過
4. 重構 **(Refactor)** — 清理程式碼，確保測試維持綠色
5. 提交 **PR** — CI 自動執行 lint + test + coverage check
6. **Code Review** — 至少一人審查 (如為個人專案則自己 review checklist)
7. **Merge** 到 **develop** — 觸發 integration test
8. **Release** — merge to main，打 tag，fastlane 自動部署

---

18. 完整依賴清單 (Key Dependencies)

用途	Package	說明
State	flutter_riverpod + riverpod_annotation	狀態管理 + code gen
DB	isar + isar_flutter_libs	本地加密資料庫
Network	dio + dio_smart_retry	HTTP client + 重試
Auth	local_auth	生物辨識
Camera	camera + google_ml_kit_barcode	QR 掃描
Secure Storage	flutter_secure_storage	Key 存儲
Testing	mockito + mocktail + integration_test	測試框架
CI	very_good_analysis	Lint rules
Logging	logger	結構化日誌
Navigation	go_router	宣告式路由
Charts	fl_chart	圓餅圖、長條圖、折線圖
Widget	home_widget	iOS WidgetKit + Android App Widgets
CSV	csv	CSV 產生與匯出
Share	share_plus	系統 Share Sheet 分享
Notification	flutter_local_notifications	應收/應付到期提醒

— End of Spec —